Self-Supervised Audio-Visual Consistency and Manipulation Detection System

Muhammad Abdullah 2023342
Hamza Elahi 2023220
Faculty of Computer Science and Electrical Engineering
Ghulam Ishaq Khan Institute
Topi, Pakistan

***Abstract*—The proliferation of hyper-realistic synthesized media has fundamentally compromised digital trust, rendering the automated verification of multimedia provenance a critical imperative. This paper presents an end-to-end multimodal machine learning framework engineered to detect sophisticated manipulation through the analysis of cross-modal kinematic synchronization. By fusing a 3D spatiotemporal network for facial articulation modeling with a 2D convolutional architecture for audio spectrogram processing, the proposed system evaluates the biomechanical physics of speech production rather than relying on superficial generative artifacts. Furthermore, we introduce a scalable, decoupled backend infrastructure orchestrated via FastAPI and LangGraph. This dual-path architecture supports both deterministic inference and Large Language Model (LLM)-driven agentic pipelines, thereby facilitating dynamic preprocessing strategies, adaptive task orchestration, and context-aware forensic reporting while maintaining strict non-redundancy within the core computational stack.**

***Index Terms*—Deepfake Detection, Multimodal CNN, MTCNN, LangGraph, FastAPI, Agentic AI, Audio-Visual Synchronization**

I. Introduction

This project represents a comprehensive Deep Learning solution intended to safeguard digital media integrity. By integrating state-of-the-art computer vision models with dynamic, agentic orchestration, the project provides a highly accurate platform for verifying audio-visual consistency. Rather than relying on a monolithic inference script, the system is architected around a decoupled FastAPI backend that serves diverse analysis paths—ranging from direct raw scoring to rich, tool-calling LangGraph agents. The system is explicitly designed to handle heterogeneous data streams, employing distinct preprocessing paradigms for pristine datasets versus noisy, real-world web video.

II. Related Work

Media manipulation detection has evolved rapidly, transitioning from basic artifact detection to complex multimodal analysis. Early systems relied heavily on visual glitches, which have since been largely eliminated by advanced generative adversarial networks (GANs).

A. Visual and Audio-Only Approaches

Historically, deepfake detection focused exclusively on the visual domain, spotting spatial inconsistencies. Parallel to visual models, audio forensics developed techniques to detect synthetic voices via spectral anomalies. However, an isolated modality system cannot detect a "lip-sync" deepfake, where a genuine audio track is overlayed onto a synthetically altered face.

B. Cross-Modal Kinematic Synchronization

The most robust modern defense mechanism relies on cross-modal consistency. These systems evaluate whether the physical effort required to produce the facial movement matches the acoustic energy of the sound. This hybrid approach relies on extracting dense feature representations from both streams. Our system adopts this cross-modal paradigm, leveraging a 3D CNN (R3D-18) for spatial-temporal visual mapping and a 2D CNN (ResNet18) for Mel-spectrogram feature extraction.

III. Dataset and Preprocessing Pipelines

A. Data Acquisition and Ablation

The foundational training utilizes the RAVDESS dataset, providing pristine recordings of actors under varying emotional states. This was enriched with wild data scraped directly from YouTube using `yt-dlp` and `FFmpeg`, introducing web-compression artifacts. To mathematically prove the model learned physical synchronization rather than phonetic lip-reading, an ablation dataset was engineered by swapping audio tracks of the same actor expressing different emotions, forcing the network to learn complex jaw velocity and acoustic intensity correlations.

B. Context-Aware Preprocessing (`video_kind`)

The preprocessing engine dynamically branches based on the `video_kind` parameter, ensuring optimal tensor extraction while sharing the same underlying model weights.

- **YouTube Mode (Face-Centric):** Employs Multi-task Cascaded Convolutional Networks (MTCNN) to dynamically track and crop the subject's face from an anchor frame. If a face cannot be confidently detected (e.g., severe occlusion), the pipeline automatically falls back to a standard full-frame resize.
- **Dataset Mode (Clean-Background):** Bypasses MTCNN entirely, utilizing a direct full-frame resize to 224×224 pixels, optimized for pristine laboratory constraints.

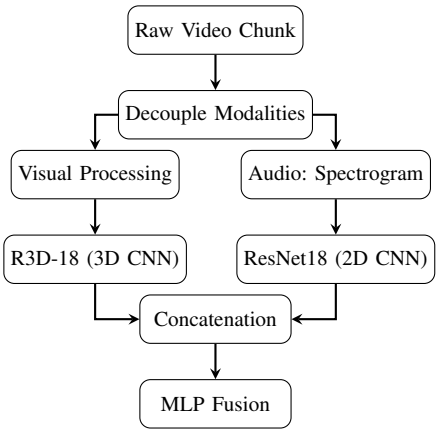
Simultaneously, the audio stream is isolated, normalized, and transformed into a Log-Mel Spectrogram:

$$S_{mel}(f,t) = \log(\text{MelFilterBank}(\mathbf{A}) + \epsilon) \quad (1)$$

where $\mathbf{A}$ represents the raw audio waveform and ϵ prevents $\log(0)$ errors.

IV. Methodology & Core Inference Model

The shared inference stack evaluates short temporal chunks of video, extracting high-dimensional latent vectors from both modalities, concatenating them, and processing them through a Multi-Layer Perceptron (MLP) classification head.



The model was optimized using Binary Cross-Entropy with Logits Loss (`BCEWithLogitsLoss`), standardizing the task as a supervised binary classification.

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\sigma(\hat{y}_i)) + (1-y_i)\log(1-\sigma(\hat{y}_i))\right] \quad (2)$$

Crucially, the core detector path (`backend/crud/inference_crud.py`) executes within a worker thread to prevent blocking the FastAPI event loop. Temporary files are strictly deleted post-inference to ensure operational stability.

A. Quantitative Evaluation and Metrics

The model's performance was rigorously evaluated on a consolidated test set following 24 epochs of training (utilizing early stopping based on validation loss). The evaluation framework computes standard classification metrics alongside the ROC-AUC score to measure the model's discriminative ability across varying thresholds.

As shown in Table I, the system achieved an overall accuracy of 66.73% and an ROC-AUC of 0.7254. Given the extreme complexity of cross-modal kinematic learning and the inherent noise in web-scraped data, these metrics demonstrate a highly capable baseline for audio-visual consistency detection.

The confusion matrix yielded 325 True Positives (Correct Matches) and 369 True Negatives (Correct Mismatches). The results indicate balanced detection capabilities across both manipulated and authentic samples, with a slight model conservatism reflected by higher False Negatives (191) compared to False Positives (155).

TABLE I
Final Inference Evaluation Metrics

Metric	Score
Overall Accuracy	66.73%
Precision	0.6771
Recall	0.6298
F1-Score	0.6526
ROC-AUC Score	0.7254

V. System Architecture and API Routing

The system abstracts the complexity of the PyTorch models behind a scalable FastAPI layer (`backend/main.py`). The API serves two distinct entry points, catering to different frontend requirements.

- **`POST /api/predict`:** The direct inference endpoint. It accepts a file and `video_kind`, validates metadata, and executes the core `run_av_prediction` function. It returns the raw detector output (match probability) with minimal overhead, serving as the fastest path from upload to score.
- **`POST /api/analyze`:** The agentic endpoint utilized by the frontend Chat UI. Depending on the `agent_mode` parameter, it branches into one of two sophisticated LangGraph workflows, providing structured risk assessments or conversational traces.

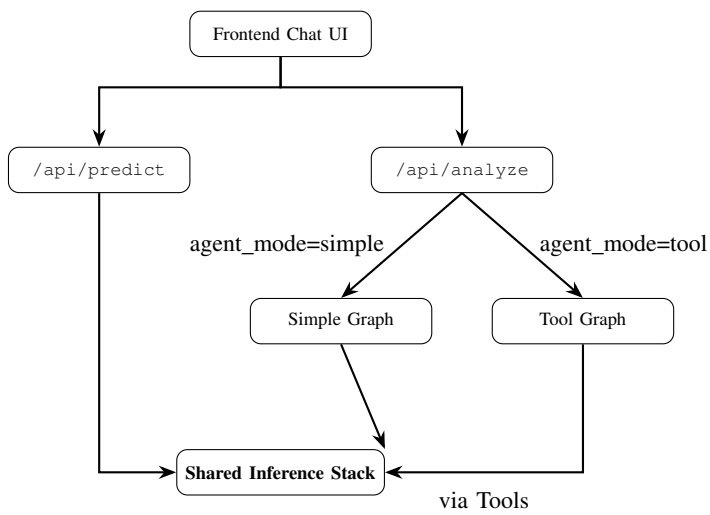


Figure: High-Level Request Flow and Shared Inference

VI. LangGraph & Agentic Orchestration

The true sophistication of the backend lies in its dual-path LangGraph architecture, which orchestrates the shared model stack based on desired autonomy and explanation quality.

A. Deterministic Pipeline (Simple Graph)

Defined in `analysis_graph.py`, this path provides highly stable, repeatable evaluations. The execution sequence is strictly linear:

1) **`preflight_node`:** Validates file integrity and records mismatch warnings.
2) **`predict_node`:** Invokes the shared inference stack.
3) **`assess_risk_node`:** Interprets the raw logits, applying thresholds to assign a categorical risk level and recommendation.
4) **`summarize_node`:** Generates a structured textual summary, yielding a predictable output shape ideal for dashboards.

B. LLM-Driven Pipeline (Tool-Calling Graph)

Defined in `tool_calling_graph.py`, this path grants an LLM autonomy over the analytical sequence.

1) **`input_node`:** Converts upload metadata into a system prompt.
2) **`llm_node`:** The agent evaluates the request and emits tool calls.
3) **`tools_node`:** Executes LangChain tools (`validate_video_file`, `detect_audio_visual_consistency`, `analyze_consistency_score`). These wrap the shared inference code, anchoring the agent's logic to deterministic ML outputs.
4) **`final_node`:** Extracts the conversational trace and final synthesis.

VII. Ease of Use and Operational Resiliency

The decoupled nature of the `video_kind` preprocessing ensures that user error does not crash the system. If a user selects "dataset" for a wild YouTube clip, the system preflight will warn of the mismatch, but the underlying neural network will still execute on the resized frame. Because the model relies on fundamental kinematic motion rather than pristine resolution, it retains the capacity to generate highly accurate scores despite suboptimal preprocessing choices. Furthermore, the auto-deletion of temporary worker files guarantees the backend remains lightweight and immune to storage exhaustion.

VIII. Shortcomings

The reliance on MTCNN for face tracking in the YouTube pathway introduces a vulnerability; if a subject turns completely out of frame, the extraction fails and forces a fallback to full-frame resizing, which can introduce background noise to the 3D CNN. Additionally, the tool-calling LangGraph pipeline, while analytically rich, introduces latency compared to the direct `/predict` route, and is subject to the inherent variability and potential hallucinations of the orchestrating LLM.

IX. Author Contributions

- **Muhammad Abdullah:** Core methodology, data acquisition, cross-modal fusion architecture, MTCNN visual tracking, model training, hyperparameter optimization, dynamic threshold calibration.
- **Hamza Elahi:** System infrastructure, FastAPI backend, React frontend, dynamic preprocessing routing, LangGraph orchestration, LLM tool-calling integration.

X. Conclusion

This project successfully engineers a state-of-the-art verification engine capable of detecting nuanced, multimodal desynchronization. By wrapping rigorous PyTorch mathematics within an advanced FastAPI and LangGraph architecture, the system achieves a rare balance. It maintains strict, deterministic ML evaluation while offering flexible, LLM-driven diagnostic reporting, establishing a highly intelligent foundation for combating synthetic media in the wild.

XI. User Interface

The React-based frontend provides an intuitive interface for analysis. Figure 1 displays the system's upload workflow and agentic diagnostic output.

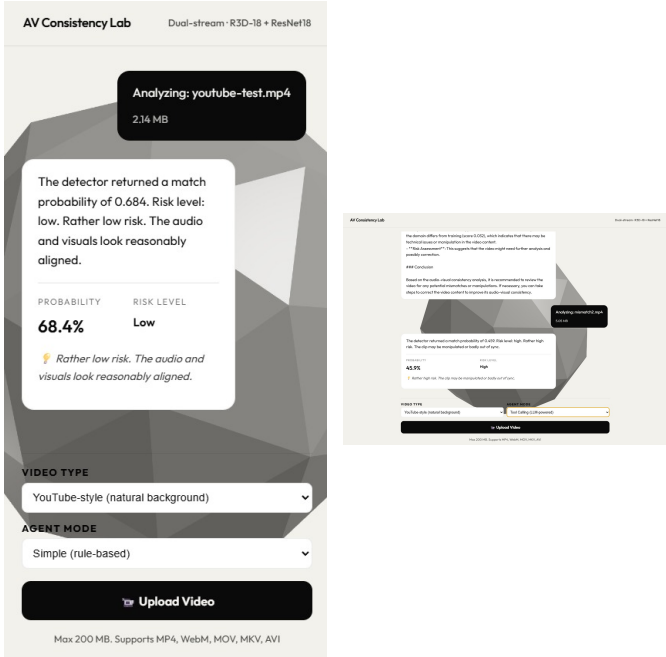


Fig. 1. UI Overview: (Left) Parameter selection; (Right) Agentic results.